



Desarrollo de un simulador de reproductor de música

Generación, almacenamiento, ordenamiento y búsqueda de canciones mediante arreglos

Integrantes:

Pablo Serón
José Cáceres
Yanira Mansilla

Fecha: 2 de octubre de 2026

Profesora: Jacqueline Aldridge
Asignatura: Estructuras de Datos

Índice

1. Introducción	2
2. Objetivo general	2
2.1. Objetivos secundarios	2
3. Organización del proyecto	3
3.1. Estructura de datos principal	3
4. Generación, validación y persistencia del catálogo	3
4.1. Generación aleatoria de datos	3
4.2. Validación	4
4.3. Mezcla del catálogo utilizando Fisher-Yates	4
4.4. CSV: Importación, exportación y arranque	4
4.5. Funciones específicas	4
4.5.1. Algoritmo de ordenamiento iterativo: Selection Sort	4
4.5.2. Algoritmo de ordenamiento recursivo: Merge Sort	5
4.5.3. Algoritmo de búsqueda binaria por recursividad	5
5. Buscar sobre una copia ordenada	5
6. Gestión del catálogo, ranking y estadísticas	6
6.1. Consultas sobre el catálogo	6
6.2. Ranking y canción más escuchada	6
7. Fila de reproducción e historial	6
7.1. Representación de la fila	6
7.2. Operaciones sobre la fila	7
7.3. Historial de las últimas K canciones	7
7.4. Reproducir una canción	7
8. Menú de terminal, errores y memoria	8
8.1. Funcionamiento del menú	8
9. Ejemplo de ejecución y pruebas	8
9.1. Salidas del programa	8
10. Conclusiones	10
11. Declaración sobre el uso de IA	11
11.1. Declaración: Pablo Serón	11
11.2. Declaración: Yanira Mansilla	11

1. Introducción

Este proyecto consistió en armar una simulación de un reproductor de música utilizando el lenguaje de programación en C. La idea de este trabajo era utilizar exclusivamente arreglos y estructuras para crear el sistema, implementando en el proceso funciones que permitiesen usar algoritmos de ordenamiento y de búsqueda. El catálogo de las canciones se debía generar de forma aleatoria y guardándose dentro de un archivo CSV una única vez para que el programa fuese capaz de leerlo y luego poder aplicar funciones relacionadas al catálogo, lista de reproducción, exportaciones y modificaciones.

En el presente informe se explicará brevemente la organización del grupo, distribución de archivos en carpetas, funcionamiento de archivo “main.c”, funciones específicas donde se refleje el funcionamiento de los algoritmos de ordenamiento y búsqueda, y salidas de datos.

2. Objetivo general

Diseñar e implementar con lenguaje C un simulador de reproductor de música. El simulador debe de ser capaz de: generar, ordenar y buscar información de canciones, utilizando únicamente arreglos.

2.1. Objetivos secundarios

1. Generar aleatoriamente un catálogo de canciones con sus atributos, validando que los valores numéricos sean coherentes con el contexto.
2. Mezclar los registros del catálogo y guardarlos en un archivo CSV que se genere una única vez.
3. Implementar algoritmo de ordenamiento iterativo y uno recursivo.
4. Implementar búsqueda binaria en su versión recursiva.
5. Desarrollar las funciones necesarias para administrar la gestión del catálogo, ranking y estadísticas, además de listar artistas, filtrar por género y obtener canciones más escuchadas.
6. Exportar catálogo actualizado al CSV tras modificar las reproducciones.
7. Construir un menú dentro de la terminal que permita utilizar las funcionalidades, manejar los errores y la correcta liberación de la memoria.
8. Documentar el código como fue acordado y trabajar de forma organizada utilizando un repositorio Git.

3. Organización del proyecto

El archivo `main.c` posee el flujo de interacción con el usuario: interfaz del menú principal, lectura de datos por teclado y conexión entre interacciones del menú y los módulos que implementan la lógica.

El sistema fue trabajado exclusivamente con arreglos estáticos, utilizando para indicar sus tamaños. El código se dividió en módulos, cada uno con su archivo `.c` y su cabecera `.h`, de modo que cada archivo se ocupa de una responsabilidad.

Encabezados	Descripción general
<code>canciones.h</code>	Definición de la estructura <code>Cancion</code> y constantes de tamaño.
<code>catalogo.h</code>	Generar, listar, contar y consultar canciones del catálogo.
<code>importacion.h</code>	Carga inicial del catálogo desde <code>catalogo.csv</code> .
<code>exportacion.h</code>	Exportación del catálogo a un archivo CSV.
<code>ordenamiento.h</code>	Selection sort, merge sort y búsqueda binaria.
<code>fila_de_reproduccion.h</code>	Fila de reproducción e historial de reproducidas.

Cuadro 1: Módulos utilizados

3.1. Estructura de datos principal

Cada canción proviene de un struct de tamaños fijos de nombre `Cancion`. Para la creación del catálogo, fila de reproducción, historial y ranking se utilizó este tipo de struct para formar arreglos estáticos de `Cancion`.

Listing 1: Struct `Cancion`

```

1 typedef struct _cancion {
2     int id;
3     char titulo[TITULO_MAX];
4     char artista[ARTISTA_MAX];
5     char album[ALBUM_MAX];
6     char genero[GENERO_MAX];
7     int duracion_seg;
8     int anho;
9     int num_reproducciones;
10 } Cancion;
```

4. Generación, validación y persistencia del catálogo

4.1. Generación aleatoria de datos

La función `generar_una_cancion` rellena una variable de tipo `Cancion` con datos aleatorios. A excepción del artista y género que son elegidos desde opciones predefinidas (8 artistas y 7 géneros) el título y álbum se forman combinando dos palabras al azar dentro de un arreglo de 8 palabras. La duración, año y reproducciones son números al azar a partir de rangos permitidos.

4.2. Validación

Antes de aceptar una canción, se validan estos datos mediante la función `validar_cancion` que comprueba la duración esté entre 30 y 900 segundos; que las reproducciones sean un número entre 0 y 1000000; que el año sea generado entre 1950 y 2026 y que si queda algo fuera de estos rangos, la función devuelva un 0.

4.3. Mezcla del catálogo utilizando Fisher-Yates

Al terminar de generar las canciones, se debían de desordenar mediante un algoritmo. En el caso de este proyecto se ha decidido utilizar la recomendación del algoritmo Fisher-Yates.

Este tipo de algoritmo permite barajar o mezclar al azar elementos que estén dentro de un arreglo o lista, donde recorren estos elementos y por cada i , elige un número al azar. El número al azar será el índice por el cual se intercambiará el elemento i .

Listing 2: Struct Cancion

```

1 for(int i = total - 1; i > 0; i--){
2     int j = rand() % (i + 1);
3     Cancion temp = catalogo[i];
4     catalogo[i] = catalogo[j];
5     catalogo[j] = temp;
6 }
```

4.4. CSV: Importación, exportación y arranque

El enunciado pide que el CSV se genere una sola vez y que las ejecuciones siguientes lo lean. Eso lo resuelve `iniciar_catalogo`: si `catalogo.csv` existe y contiene canciones válidas, se carga; si no existe o está vacío, se genera un catálogo de 200 canciones y se guarda inmediatamente.

Para leer cada línea se usa `fgets` y `sscanf` con el formato `%d,%63[^\,],...`, que lee texto hasta la siguiente coma y limita la cantidad de caracteres para no desbordar los campos. Si una línea no entrega exactamente 8 campos, o la canción no pasa la validación, se descarta sin detener el programa.

La exportación (`exportar_catalogo_csv`) escribe el encabezado y una línea por canción en un archivo distinto (`catalogo_actualizado.csv`), de forma que el original no se pierde. Se verifica el retorno de `fopen` y de `fclose` para detectar errores de escritura.

4.5. Funciones específicas

4.5.1. Algoritmo de ordenamiento iterativo: Selection Sort

En cada pasada se busca el elemento mínimo del tramo no ordenado y se intercambia con la primera posición de ese tramo. Realiza a lo más $n - 1$ intercambios y no necesita memoria

auxiliar. Dado que cada `Cancion` ocupa más de 200 bytes, minimizar los intercambios es una ventaja real.

```

1 void selection_sort(Cancion arr[], int n, int campo) {
2     Cancion temp;
3     if (campo < 1 || campo > 8) {
4         printf("Campo invalido\n");
5         return;
6     }
7     for (int i = 0; i < n - 1; i++) {
8         int min_index = i;
9
10        for (int j = i + 1; j < n; j++) {
11            if (Cancion_va_antes(arr[j], arr[min_index], campo)) {
12                min_index = j;
13            }
14        }
15        temp = arr[i];
16        arr[i] = arr[min_index];
17        arr[min_index] = temp;
18    }
19 }

```

4.5.2. Algoritmo de ordenamiento recursivo: Merge Sort

Divide el arreglo en dos mitades, ordena cada una recursivamente y las une con la función `merge`. El caso base es el tramo de uno o ningún elemento. `mergeSort` reserva un único arreglo auxiliar con `malloc`, lo reutiliza en todas las llamadas y lo libera al terminar. El punto medio se calcula como $izq + (der - izq) / 2$ para evitar desbordamiento de enteros.

```

1 void mergeSort_recursivo(Cancion arr[], Cancion aux[],
2                          int izq, int der, int campo) {
3     if (izq >= der) return; /* caso base */
4     int medio = izq + (der - izq) / 2;
5     mergeSort_recursivo(arr, aux, izq, medio, campo);
6     mergeSort_recursivo(arr, aux, medio + 1, der, campo);
7     merge(arr, aux, izq, medio, der, campo);
8 }

```

4.5.3. Algoritmo de búsqueda binaria por recursividad

Se implementaron dos versiones: `busqueda_binaria_id` y `busqueda_binaria_titulo_artista`. Ambas comparan con el elemento central y descartan la mitad donde el valor no puede estar; el caso base ($izq > der$) devuelve -1 . Cada llamada reduce el problema a la mitad.

```

1 int busqueda_binaria_id(Cancion arr[], int izq, int der, int x) {
2     if (izq > der) return -1;
3     int mid = izq + (der - izq) / 2;
4     if (arr[mid].id == x) return mid;
5     if (arr[mid].id > x) return busqueda_binaria_id(arr, izq, mid - 1, x);
6     return busqueda_binaria_id(arr, mid + 1, der, x);
7 }

```

5. Buscar sobre una copia ordenada

La búsqueda binaria exige que el arreglo esté ordenado por el campo buscado, y el usuario puede haber dejado el catálogo ordenado por cualquier otro. En lugar de reordenar el catálogo

original, la función `buscar` de `main.c` crea una copia con `duplicar_catalogo`, la ordena con Merge Sort por el campo correspondiente, busca sobre ella y libera la memoria con `free`.

6. Gestión del catálogo, ranking y estadísticas

6.1. Consultas sobre el catálogo

Las consultas de listado se resolvieron recorriendo el arreglo del catálogo y comparando textos con `strcmp`. Para `listar_artistas` y `listar_generos` se mantiene un segundo arreglo con los nombres ya encontrados: antes de copiar un nombre, se revisa si ya está en ese arreglo, de modo que cada artista o género aparece una sola vez. La función se detiene al llenar la capacidad que recibe como parámetro, así nunca escribe fuera del arreglo de salida.

Para los géneros hay tres funciones con propósitos distintos: `contar_por_genero` devuelve cuántas canciones pertenecen a un género, `listar_por_genero` guarda en un arreglo de enteros las *posiciones* del catálogo donde aparece ese género, y la cantidad por cada género del menú se obtiene combinando `listar_generos` con `contar_por_genero`. Guardar posiciones y no copias evita duplicar canciones en memoria; `main.c` usa esas posiciones para mostrar cada canción directamente desde el catálogo. De forma similar, `obtener_canciones_artista` recorre el catálogo e imprime los títulos del artista ingresado, avisando si no hay ninguno.

6.2. Ranking y canción más escuchada

Ambas funciones necesitan el catálogo ordenado por reproducciones, pero el usuario no debe perder el orden que tenía. Por eso `main.c` les entrega una copia hecha con `duplicar_catalogo`, que se libera al terminar. Con la copia ordenada de menor a mayor (campo 4, Selection Sort), el ranking se muestra recorriendo el arreglo desde el final, ya que ahí están las canciones más escuchadas. Antes de ordenar se valida que el valor de n pedido esté entre 1 y la cantidad de canciones.

La canción más escuchada por artista o por género usa la misma idea: con la copia ordenada, se recorre desde el final y la primera canción que coincide con el nombre buscado es, por construcción, la de mayor cantidad de reproducciones. Si se llega al inicio sin coincidencias, se informa que el artista o género no fue encontrado.

7. Fila de reproducción e historial

7.1. Representación de la fila

La fila es un arreglo de `Cancion` con capacidad `MAX_CANCIONES`. Como no se pueden usar otras estructuras, las casillas libres se marcan con `id = -1`, valor que ninguna canción real puede tener porque los identificadores parten en 1. La función `vaciar_fila` deja todas las

casillas con ese valor, y el resto de las funciones se detienen al encontrar la primera casilla marcada, por lo que las canciones siempre quedan contiguas al inicio del arreglo.

7.2. Operaciones sobre la fila

- **Añadir al comienzo:** `anadir_cancion` rechaza la operación si la fila está llena o si la canción ya está (`detectar_id`). En caso contrario desplaza todos los elementos una posición a la derecha y escribe la nueva canción en la posición 0. Así la canción agregada más recientemente es la primera en reproducirse.
- **Quitar:** `buscar_indice_por_id` entrega la posición de la canción y `eliminar_cancion_por_id` desplaza hacia la izquierda los elementos siguientes para tapar el hueco, marcando la última casilla como vacía. Cuando el usuario elige quitar por posición, `main.c` primero obtiene la canción que está en esa posición y luego reutiliza la misma función por ID.
- **Consultar y vaciar:** `consultar_fila` imprime las canciones hasta la primera casilla vacía (o avisa que la fila está vacía) y `vaciar_fila` vuelve a marcar todas las casillas.

7.3. Historial de las últimas K canciones

El historial es otro arreglo de `Cancion` que usa el mismo marcador de casilla vacía y que solo considera las primeras K posiciones ($K = 10$ en `main.c`). La canción más reciente siempre queda en la posición 0. Mientras haya espacio, se desplazan los elementos existentes hacia la derecha; cuando el historial está lleno, el desplazamiento sobrescribe la última posición, con lo que se descarta la canción más antigua y entra la nueva, tal como pide el enunciado.

```

1 void agregar_a_historial(Cancion historial[], Cancion cancion, int k) {
2     for (int i = 0; i < k; i++) {
3         if (historial[i].id == -1) {          /* hay hueco en i */
4             for (int j = i; j > 0; j--)
5                 historial[j] = historial[j - 1];
6             historial[0] = cancion;
7             return;
8         }
9     }
10    /* historial lleno: se pierde la mas antigua */
11    for (int i = k - 1; i > 0; i--)
12        historial[i] = historial[i - 1];
13    historial[0] = cancion;
14 }

```

7.4. Reproducir una canción

La opción de reproducir toma la primera canción de la fila, la muestra por pantalla y aumenta su contador de reproducciones. Luego la agrega al historial, actualiza la misma canción en el catálogo con `incrementar_reproducciones` y finalmente la quita de la fila. Para ubicar la canción en el catálogo se usa una búsqueda lineal por ID y no una binaria, porque el catálogo puede estar ordenado por cualquier campo en ese momento y ordenarlo de

nuevo solo para esta operación no se justifica. Gracias a esto, la exportación posterior refleja las reproducciones realizadas durante la sesión.

8. Menú de terminal, errores y memoria

8.1. Funcionamiento del menú

El `main` inicializa la semilla aleatoria, carga o genera el catálogo con `iniciar_catologo`, vacía la fila y el historial, y entra en un ciclo que muestra el menú, lee una opción y la despacha con un `switch`. Cada opción llama a una función pequeña (`buscar`, `estadisticas`, `reproducir`, etc.) que se encarga de pedir los datos y llamar a los módulos correspondientes, de manera que el `switch` se mantiene corto y legible. La lectura por teclado se hace siempre con `fgets`: `leer_entero` convierte el texto con `strtol` y entrega `-1` si no se ingresó ningún dígito, y `leer_texto` permite ingresar nombres con espacios y elimina el salto de línea. Se evitó `scanf` para que una entrada incorrecta no deje caracteres pendientes en el búfer.

Opciones	Grupo	Acción
1–5, 18	Catálogo	Generar, listar artistas, cantidades y listados por género, mostrar catálogo.
6–8	Ordenamiento y búsqueda	Ordenar por un campo con Selection o Merge Sort, buscar por ID, título o artista, canciones de un artista.
9–10	Estadísticas	Ranking de las n más escuchadas y más escuchada por artista o género.
11–14	Fila	Ver, añadir, quitar por posición o ID, y vaciar.
15–16	Historial	Ver historial y reproducir.
17	Exportación	Guardar el catálogo en <code>catalogo_actualizado.csv</code> .

Cuadro 2: Agrupación de las opciones del menú

9. Ejemplo de ejecución y pruebas

9.1. Salidas del programa

El siguiente ejemplo muestra el menú principal y el resultado de pedir el ranking de las tres canciones más escuchadas (opción 9) con el catálogo de 200 canciones generado en una ejecución real del programa.

```
===== REPRODUCTOR (200 canciones) =====
 1 Generar catalogo      2 Listar artistas      3 Cantidad por genero
 4 Listar por genero     5 Contar un genero     6 Ordenar catalogo
 7 Buscar cancion        8 Canciones de artista 9 Ranking top N
10 Mas escuchada        11 Ver fila            12 Anadir a fila
13 Quitar de fila        14 Vaciar fila          15 Ver historial
16 Reproducir           17 Exportar CSV         18 Mostrar catalogo
 0 Salir
Opcion: 9
Top N: 3
```

Top 3 de Canciones con mas reproducciones:

1)	Fuego Noche		Canserbero		998036 reproducciones
2)	Luna Ritmo		Milo_J		981660 reproducciones
3)	Camino Ritmo		Los_Charros_de_Lumaco		981158 reproducciones

En otra ejecución se añadieron a la fila las canciones con ID 5 y 9, se intentó añadir de nuevo la 5 y se reprodujo la primera de la fila. El programa rechazó el duplicado con el mensaje “Canción ya esta en la fila de reproducción”, la fila quedó con la 9 en primer lugar (por insertarse al comienzo), y al reproducirla la canción salió de la fila y pasó a ser la primera del historial.

1	9 Silencio Ritmo Lit_Killah
2	5 Sombra Noche Duki

10. Conclusiones

Con este proyecto se logró tener un simulador de reproductor musical que cumple con las funcionalidades pedidas y que es controlable mediante la terminal. El trabajar con arreglos nos ha ayudado a reforzar el desplazamiento de elementos, control de límites y la liberación de memoria. Además, nos permitió tener un conocimiento práctico respecto a las diferencias de comportamiento que hay entre un ordenamiento iterativo y uno recursivo, incluyendo a la búsqueda binaria recursiva.

Igualmente, este proyecto nos ha ayudado a aprender a organizarnos como grupo para la realización de tareas, qué precauciones tener a nivel grupal y la comunicación constante como grupo para un trabajo mucho más eficiente.

Lo más complicado en este proyecto fue saber dónde podríamos integrar algoritmos y analizar si el caso requería de ello o si era posible de simplificar alguna otra manera.

11. Declaración sobre el uso de IA

11.1. Declaración: Pablo Serón

Durante el desarrollo de mi parte del proyecto (gestión de catálogo, exportación e importación de datos) hice uso de la herramienta Claude. Esto fue con el propósito de resolución de dudas conceptuales como: include guards(ifndef/endif), depuración de errores tanto para código dentro de catalogo.c al compilar y errores de interconexion de diversos archivos ya en el main.c y por último para comprender el funcionamiento del algoritmo de mezcla Fisher-Yates.

Declaración: José Cáceres Se utilizó IA como apoyo para aclarar ciertos conceptos relacionados con recursividad, revisar y depurar errores del código, mejorar los comentarios y documentación dentro de las funciones de búsqueda y ordenamiento, y apoyarse en la organización de las funciones dentro del main.c

11.2. Declaración: Yanira Mansilla

Durante el desarrollo al crear el encabezado y el .c de `fila_de_reproduccion` se ocupó la herramienta Claude como apoyo para la depuración de errores, para coprimir código si se veía que estaba siendo redundante y para resolver dudas conceptuales.